

# SPRING AI

**Manoel C M Neto**

# PREPARANDO O AMBIENTE: CADASTRO NA OPENAI

- Ao longo do curso vamos escrever uma aplicação Java que se integra com a API da OpenAI
- Vamos fazer requisições para as funcionalidades de geração de textos e imagens via inteligência artificial generativa
- Será necessário a criação de uma conta na OpenAI, caso você ainda não tenha criado

# PREPARANDO O AMBIENTE: CADASTRO NA OPENAI

- Passo 1: Acesso ao site da OpenAI:  
<https://platform.openai.com/login>
- Passo 2: Criação da conta
  - Faça o login usando uma das opções (eu uso google)
- Passo 3: Usando o dashboard
  - Depois de configurar seu perfil, você será redirecionado para a página do dashboard da OpenAI, onde você pode explorar várias opções e serviços oferecidos pela OpenAI
- Passo 4: método de pagamento
  - Para utilizar a API da OpenAI e acessar todos os recursos, você precisará adicionar um método de pagamento. Acesse a página de Billing e clique em "Payment Methods" para vincular o seu cartão de crédito

# PREPARANDO O AMBIENTE: CADASTRO NA OPENAI

- Passo 5: Adicionando crédito à sua conta
  - Após cadastrar o cartão de crédito, será necessário adicionar um valor para ser utilizado em sua conta. Volte para a página de Billing e clique no botão "Add to credit balance".
  - O valor mínimo que a OpenAI permite é de 5 dólares, algo mais do que suficiente para o que utilizaremos ao longo do curso.

# CRIANDO UM PROJETO SPRING BOOT

- Para demonstrar o uso do Spring AI vamos criar uma API REST clássica em Spring Boot.
- Vá ao site do spring inicializr e crie um projeto usando a versão 3 do Spring
- Adicione as dependências WEB e SPRING AI (OpenAI e não Azure OpenAI)
- Gere e baixe o projeto, descompacte e em seguida use o IDE da sua preferência para importar o projeto Maven.

# INTEGRAÇÃO COM A OPENAI

- O objetivo do nosso projeto é pedir para IA sugerir a nomes para produtos ecológicos.
- É como se você fosse ao chatGPT e pedisse: Crie 5 produtos ecológicos!
- Para começar, na sua aplicação crie uma classe "GeradorDeProdutosController" (claro que num package controller)
-

# INTEGRAÇÃO COM A OPENAI

- // código omitido
- 
- @RestController
- @RequestMapping("gerador")
- public class GeradorDeProdutosController {
- 
- @GetMapping
- public String gerarProdutos() {
- return "";
- }
- 
- }
-

# INTEGRAÇÃO COM A OPENAI

- Vamos inicialmente usar o "ChatClient API" do Spring AI
- Para isso precisamos importar "ChatClient" do `org.springframework.ai.chat.client.ChatClient;`
- Instanciar ele usando um Builder no construtor:
- `private final ChatClient chatClient;`
- 
- `public GeradorDeProdutosController(ChatClient.Builder chatClientBuilder) {`
- `this.chatClient = chatClientBuilder.build();`
- `}`



# INTEGRAÇÃO COM A OPENAI

```
• @RestController
• @RequestMapping("gerador")
• public class GeradorDeProdutosController {
•
•     private final ChatClient chatClient;
•
•     public GeradorDeProdutosController(ChatClient.Builder chatClientBuilder) {
•         this.chatClient = chatClientBuilder.build();
•     }
•
•     @GetMapping
•     public String gerarProdutos() {
•         var pergunta="Gere 5 produtos ecologicos";
•         return this.chatClient.prompt()
•             .user(pergunta)
•             .call()
•             .content();
•     }
• }
```

# INTEGRAÇÃO COM A OPENAI

- A chamada do **chatClient** é bem simples:
  - o que você faz é basicamente criar um prompt
    - `.user` você passa a entrada do usuário
    - `.call` faz a chamada da IA e
    - `.content` retorna uma string com o resultado
- Antes de testar o projeto, é preciso passar as credenciais de acesso a OpenIA para o aplicativo

# API KEY E VARIÁVEIS DE AMBIENTE

- Se você rodar o projeto como ele está, no console, na linha que contém "Caused by" e mostra a causa raiz da exceção, é indicado que a chave da API da OpenAI deve ser configurada ("OpenAI API key must be set.").
- Primeiramente, em projetos com Spring Boot, isso deve ser configurado no arquivo **application.properties**
- Vamos usar spring.ai.openai.api-key
- Para criar uma API KEY vá no dashboard da OpenIA
- Lá acesse API Keys. Em seguida Create new secret Key

# API KEY E VARIÁVEIS DE AMBIENTE

- Lá acesse API Keys. Em seguida Create new secret Key
- Em seguida dê um nome a sua chave e escolha o projeto default.
- Na sequência copie logo a chave pois se você fechar a janela terá que apagar e criar uma nova chave
- agora no spring.ai.openai.api-key=<cole aqui a chave>
- **OBSERVAÇÃO: NÃO É UMA BOA PRÁTICA COLOCAR A CHAVE NO CORPO DO PROJETO!!!**

# API KEY E VARIÁVEIS DE AMBIENTE

- No eclipse, clique com o botão direito no projeto.
- Run As → Run Configurations...
- Selecione sua aplicação Spring Boot.
- Aba Environment.
- Clique em New.
- Preencha o Nome:API\_KEY e o Valor:<Cole a chave>
- Em **spring.ai.openai.api-key=\${API\_KEY}**
- **Agora pode rodar a sua API e fazer uma requisição get no navegador e observar o resultado!!!**

# ANÁLISE DE SENTIMENTOS

- Vamos imaginar o cenário hipotético: A empresa Tech4You está desenvolvendo um sistema de análise de sentimentos para reviews de produtos.
- O objetivo é identificar automaticamente se um comentário expressa um sentimento positivo, negativo ou neutro.
- Você, como pessoa desenvolvedora da Tech4You, recebeu a tarefa de integrar a API da OpenAI ao sistema, utilizando Spring AI.
- O código do Slide a seguir representa a implementação inicial do endpoint responsável por receber o texto da review e retornar o sentimento analisado

# ANÁLISE DE SENTIMENTOS

- @RestController
- @RequestMapping("reviews")
- public class AnaliseSentimentosController {
- 
- private final ChatClient chatClient;
- 
- public AnaliseSentimentosController(ChatClient.Builder chatClientBuilder) {
- this.chatClient = chatClientBuilder.build();
- }
- 
- @PostMapping("/analisar")
- public String analisarSentimento(@RequestBody String review) {
- // Implementação da chamada à API da OpenAI
- }
- }

# ANÁLISE DE SENTIMENTOS

- `@PostMapping("/analisar")`
- `public String analisarSentimento(@RequestBody String review) {`
- `var pergunta = String.format("Analise o sentimento da seguinte review: %s. Responda com 'Positivo', 'Negativo' ou 'Neutro'.", review);`
- 
- `return this.chatClient.prompt()`
- `.user(pergunta)`
- `.call()`
- `.content();`
- `}`



# PLAYGROUND E PARÂMETROS

- Acesse <https://platform.openai.com/playground>
- Lá você pode alterar parâmetros de uso da OpenAI
- A API da OpenAI possui diversos parâmetros disponíveis que podem ser ajustados para obter os resultados desejados em suas aplicações. Esses parâmetros influenciam o comportamento do modelo e a qualidade das respostas geradas. Veja a seguir os principais parâmetros e como eles funcionam:

# PLAYGROUND E PARÂMETROS

- Temperature:

- Definição: O parâmetro Temperature controla a aleatoriedade das respostas geradas pelo modelo.
- Intervalo de valores: 0.0 a 2.0
- Descrição: Valores mais baixos, como 0.5 ou 0.6, fazem com que o modelo produza respostas mais determinísticas e focadas, reduzindo a criatividade. Já valores mais altos, como 1.2 ou 1.5, aumentam a criatividade e a diversidade das respostas, mas também podem introduzir mais incoerência.
- Uso: Use um valor baixo para tarefas que requerem respostas precisas e coerentes, como explicações técnicas. Use temperature alta para tarefas criativas, como escrita de histórias.

# PLAYGROUND E PARÂMETROS

- Max Tokens

- Definição: O parâmetro Maximum Tokens define o número máximo de tokens (palavras ou partes de palavras) na resposta gerada pelo modelo.
- Intervalo: Depende do modelo utilizado. Normalmente tem um limite de 4096 ou 16384.
- Descrição: Este parâmetro limita o comprimento da resposta. Por exemplo, se for definido como 50, a resposta não ultrapassará 50 tokens.
- Uso: Ajuste de acordo com a necessidade da aplicação. Para respostas curtas e diretas, use um valor menor. Para respostas detalhadas, use um valor maior.

# PLAYGROUND E PARÂMETROS

- TopP

- Definição: O parâmetro TopP, também conhecido como núcleo de amostragem, controla a diversidade através da restrição do conjunto de candidatos para a próxima palavra.
- Intervalo: 0.0 a 1.0
- Descrição: Com TopP de 0.9, o modelo considera apenas os 90% dos candidatos mais prováveis (com base na sua probabilidade cumulativa) para a próxima palavra. Isso pode tornar as respostas mais variadas e menos previsíveis.
- Uso: Use em conjunto com temperature para ajustar a criatividade e diversidade das respostas. Valores mais altos podem gerar respostas mais variadas.

# PLAYGROUND E PARÂMETROS

- Frequency penalty
  - Definição: O parâmetro Frequency penalty aplica uma penalidade a palavras que já apareceram na resposta.
  - Intervalo: 0.0 a 2.0
  - Descrição: Valores positivos reduzem a probabilidade de repetir as mesmas palavras, incentivando a diversidade lexical. Valores negativos podem aumentar a probabilidade de repetição.
  - Uso: Utilize um valor alto quando quiser respostas com menor repetição de palavras, sendo útil em situações que requerem maior variação lexical

# PLAYGROUND E PARÂMETROS

- Presence penalty

- Definição: O parâmetro Presence penalty penaliza a presença de determinadas palavras na resposta.
- Intervalo: 0.0 a 2.0
- Descrição: Similar ao Frequency penalty mas com foco na presença de palavras, não apenas na frequência. Valores altos desencorajam a inclusão de palavras presentes em contextos anteriores, enquanto valores baixos incentivam essa inclusão.
- Uso: Ajuste para controlar a inclusão de certas palavras na resposta, mantendo ou alterando o contexto de forma mais flexível.

# PLAYGROUND E PARÂMETROS

- Stop sequences

- Definição: O parâmetro Stop sequences define uma ou mais sequências de caracteres que, ao serem encontradas, fazem com que o modelo interrompa a geração de texto.
- Descrição: Pode ser uma string única ou uma lista de strings. Quando uma dessas sequências aparece na resposta gerada, a geração de texto é interrompida.
- Uso: Use para delimitar respostas e evitar saídas excessivamente longas ou fora de contexto. Por exemplo, em um chatbot, você pode definir sequências de parada para garantir que o modelo responda de maneira concisa.

# CATEGORIZADOR DE PRODUTOS

- Vamos agora criar um outro controlador (chamado CategorizadorDeProdutosController) para categorizar produtos usando parâmetros da OpenIA
- @GetMapping
- public String categorizar(String produto) {
- var system = "Você é um categorizador de produtos";
- return this.chatClient.prompt()
- .system(system)
- .user(produto)
- .options(**OpenAiChatOptions**.builder()
- **.temperature(0.85)**
- .build())
- .call()
- .content();
- }



# CATEGORIZADOR DE PRODUTOS

- Abra um navegador e digite:
  - <http://localhost:8080/categorizador?produto=Celular>
  -

# PROMPT ENGINEERING

- No "campo system" nós fomos bem genéricos ao informar o contexto:
  - `var system = "Você é um categorizador de produtos";`
- Vamos tentar ser mais específicos e ver o que acontece:
  - `var system = "Você é um categorizador de produtos e deve retornar apenas o nome da categoria do produto informado";`
- Ao rodar a requisição, perceba que o sistema retorna apenas o nome da categoria

# PROMPT ENGINEERING

- Podemos ser ainda mais específicos:
  - `var system = ""Você é um categorizador de produtos e deve responder apenas o nome da categoria do produto informado`
  - 
  - Escolha uma categoria dentro a lista abaixo:
  - 
  - Higiene pessoal
  - 
  - Eletronicos
  - 
  - Esportes
  - 
  - Outros
  - 
  - exemplo de uso:
  - Pergunta: Bola de futebol
  - 
  - Resposta: Esportes"";

# PROMPT ENGINEERING

- Essa prática de tentar dar o máximo de informação de contexto para o prompt é chama de "**Prompt Engineering**"
- Ao construir prompts eficazes, você pode direcionar o comportamento do modelo e obter respostas mais relevantes e precisas.
- A seguir listamos algumas boas práticas de prompt engineering:

# PROMPT ENGINEERING

- **Escreva instruções claras e diretas:** Quanto mais objetivas forem as instruções, mais preciso será o resultado. Evite ambiguidade e forneça exemplos explícitos se necessário.
- **Faça perguntas diretas:** Formule perguntas de maneira direta para obter respostas diretas. Por exemplo, ao invés de perguntar "O que você acha sobre o clima?", pergunte "Qual é a previsão do tempo para amanhã em Maceió?".

# PROMPT ENGINEERING

- **Forneça contexto:** Incluir contexto adicional no prompt pode ajudar o modelo a entender melhor a pergunta e a fornecer uma resposta mais precisa. Por exemplo, "Explique a teoria da relatividade como se eu fosse um estudante do ensino médio".
- **Use formatação estrutural:** Utilizar listas, títulos e seções pode ajudar a guiar o modelo na formatação da resposta. Por exemplo, "Liste três benefícios do exercício físico".
- **Defina o estilo da resposta:** Indique o tom e o estilo que você deseja na resposta. Por exemplo, "Responda de forma profissional e concisa:" ou "Escreva uma história infantil divertida sobre um dragão".

# PROMPT ENGINEERING

- **Exemplos e demonstrações:** Inclua exemplos de respostas desejadas para guiar o modelo. Por exemplo, "Aqui estão alguns exemplos de como responder a perguntas sobre tecnologia:" seguido de exemplos. Você também pode mostrar exemplos de respostas inadequadas para ajudar o modelo a entender o que evitar.
- **Uso de restrições e limitações:** Defina claramente os limites da resposta esperada. Por exemplo, "Forneça uma resposta com no máximo 50 palavras". Se a resposta deve ser contida dentro de um determinado escopo, especifique-o. Por exemplo, "Explique apenas os benefícios ambientais do uso de energia solar".

# DIFERENÇA ENTRE MODELOS

- Cada modelo da OpenAI ( e dos seus concorrentes) tem características/limites específicas, exemplo (em 10/12/2026):
- <https://developers.openai.com/api/docs/models>
  - **0 GPT-4o** suporta uma janela de contexto de 128.000 tokens e gera uma saída máxima de 4.096 tokens além de ser treinados com dados até 10/2023. O custo desse modelo é U\$5 para cada 1M de tokens de entrada
  - **0 GPT-5.4 mini** entre outras diferenças tem preço U\$0.75 para cada 1M de tokens de entrada
  - **0 GPT-5.5** suporta uma janela de contexto de 1M tokens e gera uma saída máxima de 128K tokens além de ser treinados com dados até 12/2025. O custo desse modelo é U\$5 para cada 1M de tokens de entrada



# MAS O QUE SÃO TOKENS?

- Modelos de linguagem generativa, como o GPT-5.5, estão na frente da tecnologia de processamento de linguagem natural (NLP).
- Esses modelos são capazes de gerar texto, traduzir idiomas, escrever ensaios e até mesmo responder perguntas de maneira coerente e contextualmente relevante.
- Para fazer isso de forma eficaz, eles precisam processar grandes quantidades de dados textuais, e é justamente aqui que os tokens entram em cena.

# MAS O QUE SÃO TOKENS?

- Para tratar texto de maneira computacional, é necessário dividi-lo em unidades menores e mais fáceis de manipular.
- Porém, executar essa divisão de forma que o modelo possa entender e gerar texto sem perder o contexto ou a qualidade é um desafio.
- Uma forma eficaz de abordar esse problema é por meio dos tokens.

# MAS O QUE SÃO TOKENS?

- **Tokens** são as unidades básicas de texto que um modelo de linguagem utiliza para processar e gerar informações.
- Eles são como "blocos de construção" que compõem palavras, frases e sentenças.
- Dependendo da implementação, um token pode ser:
  - **Palavras inteiras:** Em alguns modelos, cada palavra pode ser tratada como um token;
  - **Sub-palavras:** Em modelos mais avançados, palavras podem ser divididas em partes menores chamadas sub-palavras ou morfemas;
  - **Caracteres individuais:** Em alguns casos, caracteres individuais de uma palavra podem ser considerados tokens, embora isso seja menos comum.

# COMO OS TOKENS FUNCIONAM?

- Quando você insere texto em um modelo de linguagem, o texto é primeiro tokenizado, ou seja, dividido em tokens.
- Esses tokens são então convertidos em números, ou vetores, que o modelo pode processar.
- Aqui está um exemplo prático:
  - <Slide a seguir>

# COMO OS TOKENS FUNCIONAM?

- **Entrada de texto:** "O sol está brilhando."
- **Tokenização:**
  - "O"
  - "sol"
  - "está"
  - "brilhando"
- **Conversão para vetores:** Cada token é convertido em uma representação numérica que o modelo pode interpretar.
- **Processamento:** O modelo processa esses vetores para gerar a resposta desejada.
- **Geração de texto:** A resposta do modelo é convertida de volta de vetores para tokens e, então, para texto legível.

# COMO OS TOKENS FUNCIONAM?

- Entender como os tokens funcionam é importante, pois isso pode influenciar como você utiliza as ferramentas de IA generativa. Por exemplo:
  - **Limitação de comprimento:** Os modelos de linguagem têm uma limitação de quantos tokens podem processar de uma vez. Isso significa que prompts e respostas combinados não podem exceder esse limite.
  - **Custo e eficiência:** O processamento de tokens é uma das bases para calcular os custos ao usar a API da OpenAI. Mais tokens significam mais processamento e, portanto, maior custo.
  - **Qualidade da resposta:** A forma como o texto é tokenizado pode afetar a qualidade da resposta gerada pelo modelo. Uma tokenização eficaz preserva o significado e o contexto, resultando em respostas mais precisas e relevantes.

# PROCESSAMENTO EM LOTE

- A OpenAI oferece uma poderosa ferramenta para lidar com grandes volumes de dados de forma eficiente e econômica: a Batch API.
- Com ela, você pode processar tarefas como geração de texto, tradução e análise de sentimentos em lote, sem comprometer o desempenho ou os custos.
- Enquanto algumas aplicações exigem respostas imediatas da API da OpenAI, muitas vezes você se depara com a necessidade de processar grandes conjuntos de dados que não requerem resposta em tempo real.
- É aí que a Batch API se destaca. Imagine, por exemplo, classificar milhares de documentos, gerar embeddings para um repositório inteiro de conteúdo ou realizar análises de sentimentos em grandes quantidades de avaliações de clientes.

# PROCESSAMENTO EM LOTE

- Com a Batch API, em vez de enviar milhares de requisições individuais, você as agrupa em um único arquivo e envia para a API.
- Isso traz diversas vantagens. Primeiramente, você obtém um desconto de 50% nos custos em comparação com o envio de requisições individuais.
- Além disso, a Batch API possui limites de taxa de uso (rate limits) significativamente maiores, permitindo que você processe muito mais dados em um período menor.



# PROCESSAMENTO EM LOTE

- A Batch API é compatível com diversos modelos, incluindo os mais antigos ainda em uso e modelos de embedding de texto. Visite:
- <https://developers.openai.com/api/docs/guides/batch>
-

# COMO ESCOLHER O MODELO VIA PROGRAMAÇÃO?

- Podemos mudar o modelo em alguns locais no Spring. Uma delas é na própria requisição:
- @GetMapping
- ```
    public String categorizar(String produto) {
```
- ```
        var system = "Você é um categorizador de produtos";
```
- ```
        return this.chatClient.prompt()
```
- ```
            .system(system)
```
- ```
            .user(produto)
```
- ```
            .options(OpenAiChatOptions.builder()
```
- ```
                .temperature(0.85)
```
- ```
                .model("gpt-5.4-mini")
```
- ```
                .build())
```
- ```
            .call()
```
- ```
            .content();
```
- ```
    }
```
- 
-

# COMO ESCOLHER O MODELO VIA PROGRAMAÇÃO?

- Podemos mudar o modelo na criação do `chatClient` no construtor. Nesse caso todas as requisições vão usar o mesmo modelo:

- `private final ChatClient chatClient;`
- `public CategorizadorDeProdutosController(ChatClient.Builder chatClientBuilder) {`
- `this.chatClient = chatClientBuilder`
- `.defaultOptions(OpenAiChatOptions.builder()`
- `.model("gpt-5.4-mini")`
- `.build())`
- `.build();`
- `}`

# COMO ESCOLHER O MODELO VIA PROGRAMAÇÃO?

- Podemos também incluir no `application.properties` uma variável escolhendo o modelo:
- **spring.ai.openai.chat.options.model=gpt-5.4-mini**
- Nesse caso todas as requisições usam esse modelo.
- Se você não configurar nada, por padrão ( na versão do Spring AI 1.1.7) o modelo é o `gpt-4o-mini`

# INJEÇÃO DINÂMICA DO MODELO

- Ao utilizar a API da OpenAI em uma aplicação Spring Boot, você pode se deparar com a necessidade de interagir com diferentes modelos, como o "gpt-4o" para tarefas mais complexas e o "gpt-4o-mini" para respostas mais rápidas e econômicas.
- Definir múltiplos ChatClients diretamente no Controller pode tornar o código complexo e difícil de gerenciar, principalmente à medida que a aplicação cresce e novas funcionalidades são adicionadas.
- Imagine ter que replicar a configuração do ChatClient em diversos Controllers, cada um com um modelo específico.

# INJEÇÃO DINÂMICA DO MODELO

- Uma abordagem mais organizada e eficiente é criar uma classe de configuração separada, onde definimos cada ChatClient como um Bean gerenciado pelo Spring.
- Você pode criar um @Configuration e lá definir @Bean com cada modelo desejável.
- Depois é só escolher qual deles usar no código
- Veja a seguir um exemplo em código de como implementar isso:

# INJEÇÃO DINÂMICA DO MODELO

- **@Configuration**
- `public class ChatClientConfiguration {`
- 
- **@Bean**
- **@Qualifier("gpt-4o-mini")**
- `public ChatClient gpt4oMiniChatClient(ChatClient.Builder`  
`chatClientBuilder) {`
- `return chatClientBuilder`
- `.defaultOptions(OpenAiChatOptions.builder()`
- `.model("gpt-4o-mini")`
- `.build())`
- `.build();`
- `}`

# INJEÇÃO DINÂMICA DO MODELO

- **@Bean**
- **@Qualifier("gpt-4o")**
- `public ChatClient gpt4oMiniChatClient(ChatClient.Builder chatClientBuilder) {`
- `return chatClientBuilder`
- `.defaultOptions(OpenAiChatOptions.builder()`
- `.model("gpt-4o")`
- `.build())`
- `.build();`
- `}`
- 
- 
-



# INJEÇÃO DINÂMICA DO MODELO

- Agora é só escolher e usar ... como por exemplo:
  - `@RestController`
  - `@RequestMapping("gerador")`
  - `public class GeradorDeProdutosController {`
  - 
  - `private final ChatClient chatClient;`
  - 
  - `public GeradorDeProdutosController(@Qualifier("gpt-4o")`  
`ChatClient chatClient) {`
  - `this.chatClient = chatClient;`
  - `}`
  - 
  - `// ... métodos do controller ...`
  - `}`

# CONTAGEM DE TOKENS

- Você pode contar os tokens de entrada para saber se a entrada é possível ou desejável.
- Assim, entradas muito grandes, se for o caso, podem ser barradas.
- Existem bibliotecas que permitem fazer isso como a JTokkit
  - inclua no `pow.xml`:
  - `<dependency>`
  - `<groupId>com.knuddels</groupId>`
  - `<artifactId>jtokkit</artifactId>`
  - `<version>1.1.0</version>`
  - `</dependency>`

# CONTAGEM DE TOKENS

- `EncodingRegistry registry =  
Encodings.newDefaultEncodingRegistry();`
- 
- `Encoding encoding =  
registry.getEncodingForModel(ModelType.GPT_40);`
- 
- `String texto = "Olá mundo!";`
- 
- `int tokens = encoding.countTokens(texto);`
- 
- `System.out.println("Tokens: " + tokens);`

# POSSÍVEIS ERROS DA API

- O uso da OpenAI API a partir na nossa API significa que estamos fazendo uma chamada a uma API externa.
- Dessa forma, podem ocorrer erros de comunicação. Veja alguns códigos de erros aqui:
  - <https://developers.openai.com/api/docs/guides/error-codes>
- Podemos fazer uso de Try/Catch ou criar um Controller Advice. Mas a boa notícia é que o próprio SpringAI tem opções para tratar esse tipo de situação.

# PARÂMETROS DE RETRY

- O framework tem o que ele chama de **retry properties**. Da mesma forma que podemos configurar uma variável para dizer qual modelo usar, podemos também fazer o mesmo para os casos de indisponibilidade da OpenAI.
- Por exemplo, a propriedade **spring.ai.retry.max-attempts** define quantas tentativas serão realizadas caso uma requisição falhe,
- Por padrão são 10 tentativas. Se todas as 10 tentativas falharem, uma exceção será lançada, indicando que a API da OpenAI está fora do ar.

# PARÂMETROS DE RETRY

- Outros parâmetros incluem o intervalo de tempo entre tentativas (**spring.ai.retry.backoff.initial-interva**), que é de 2 segundos por padrão, e o multiplicador, que inicia em 10 segundos e multiplica esse valor por 2 a cada nova tentativa.
- O parâmetro máximo é 5 nesse caso. Além disso, o intervalo máximo de tempo que essas requisições podem levar é de 3 minutos, entre outras propriedades disponíveis.

# PARÂMETROS DE RETRY

- Exemplo:
  - `spring.ai.retry.max-attempts=20`
  - `spring.ai.retry.backoff.initial-interval=10`
  - `spring.ai``.retry.backoff.max-interval=300`
- Se ainda assim a API estiver com problemas o Spring vai lançar uma exceção e você pode tratar de forma tradicional.

# GERANDO LOGS

- Como Gerar e Registrar Logs da API da OpenAI?
- Primeiro é preciso incluir a seguinte linha de exemplo:
  - `return this.chatClient.prompt()`
  - `.system(system)`
  - `.advisors(new SimpleLoggerAdvisor()).user(produto)`
  - `.options(OpenAiChatOptions.builder()`
  - `.temperature(0.85)`
  - `//.model("gpt-5.4-mini")`
  - `.build())`
  - `.call().content();`



# GERANDO LOGS

- Além disso, para que o log possa aparecer na console é preciso habilitar a propriedade: `// código omitido`
  - [logging.level.org.springframework.ai](#).chat.client.advisor=DEBUG
- Depois de rodar, observe a console da aplicação

# MODELO MULTIMODAL

- A maioria dos modelos de AI servem para outras coisas além de gerar respostas em texto ou mesmo receber entradas em texto.
- Os provedores de inteligência artificial possuem um conceito chamado de **modelo multimodal**, o que significa que eles não se limitam apenas à geração de texto.
- A OpenAI, por exemplo, oferece suporte para geração de imagens, além de outros tipos de conteúdos.

# MODELO MULTIMODAL

- Vamos ver como gerar imagens por exemplo. Para isso vamos criar um novo controller:
  - `@RestController`
  - `@RequestMapping("imagem")`
  - `public class GeradorDeImagensController {`
  - 
  - `@GetMapping`
  - `public String gerarImagem(String prompt) {`
  - 
  - `}`
  - `}`

# MODELO MULTIMODAL

- Não vamos mais usar o ChatClient. Vamos usar o ImageModel API da OpenAi
- Para isso:
  - `import org.springframework.ai.image.ImageModel;`
  - `@RestController`
  - `@RequestMapping("imagem")`
  - `public class GeradorDeImagemController {`
  - 
  - `private final ImageModel imageModel;`
  - 
  - `public GeradorDeImagemController(ImageModel imageModel) {`
  - `this.imageModel = imageModel;`
  - `}`
  - 
  - `// código omitido...`
  - `}`

# MODELO MULTIMODAL

```
• @GetMapping
•     public ResponseEntity<?> gerarImagem(String prompt) {
•         var options = OpenAiImageOptions
•             .builder()
•             .height(1024)
•             .width(1024)
•             .build();
•
•         var response = imageModel.call(new ImagePrompt(prompt, options));
•         var base64Image = response.getResult().getOutput().getB64Json();
•         byte[] imageBytes =
•             Base64.getDecoder().decode(base64Image);
•
•         return ResponseEntity.ok()
•             .contentType(MediaType.IMAGE_PNG)
•             .body(imageBytes);
•     }
```

# MODELO MULTIMODAL

- O Spring consegue injetar o ImageModel diretamente, diferente do ChatClient, onde precisávamos injetar o Builder e criar o ChatClient.
- Agora, como implementar a chamada, disparar a requisição e armazenar a imagem gerada?

# MODELO MULTIMODAL

- No método `gerarImagem()`, criaremos uma variável `response` que representará a resposta da requisição, recebendo como parâmetro o `imageModel` e o método `call()`. Ele funciona de maneira semelhante ao gerador de texto do `ChatClient`, mas recebe um parâmetro.
- Como parâmetro do método `call()`, vamos instanciar o `new ImagePrompt()`, que conterá o prompt do usuário e os parâmetros para personalizar a imagem, como tamanho, altura, largura e resolução.
- O `ImagePrompt` recebe dois parâmetros: o prompt, que será a variável dinâmica digitada pela pessoa usuária; e também as `options` de personalização

# MODELO MULTIMODAL

- Antes dessa linha, criaremos uma variável `options` para personalizar a imagem. Se não passarmos `options`, ele usará o formato padrão do Spring.
- Essa variável receberá `OpenAiImageOptions.builder().build()`. Contudo, antes de chamar o `build()`, também configuraremos os parâmetros `.height(1024)` e `.width(1024)`.
- Desse modo, teremos uma imagem com 1024 de largura por 1024 de altura. Você pode definir o tamanho mais adequado para seu projeto



# MODELO MULTIMODAL

- Em resumo, para fazer a chamada da API, usamos `imageModel.call()`, passando a string `prompt` com a mensagem da pessoa usuária sobre qual imagem gerar e as opções de personalização.
- A chamada do `call()` devolve um objeto `response`. O que retornaremos para o navegador será um `response.getResult().getOutput()`.
- Assim, obteremos a da imagem gerada em Base64.
- Para exibir a imagem é preciso decodificar de Base64 para PNG (por exemplo)
- Ou Salvar em disco para depois gerar uma URL, etc.